Development process: how this model was built

This page records how FinFraud was built, version by version. The first version was a variational autoencoder trained on two CPU cores. The current one, v7, bases its verdict on gradient-boosted decision trees trained on labelled fraud. The trees read the customer's payment history, the entities around the payment and, for two of the eleven banks, the output of a transformer pretrained on 328 million unlabelled payments; the autoencoder still runs beside them and explains what is unusual about each payment. For every version the page states what changed, what it cost and what it did not improve, and the last sections explain how to run and test the model yourself. The plain-language version covers the same ground with fewer technical terms. Terms set in slightly bold italics show a definition when you rest the pointer on them or tap them.

By Dean Mostafa. The code, the training pipeline and every pretrained version on this site are in the repository, github.com/deanOcoin/fraudvae. This site runs that repository in its public mode, which only scores payments.

1. What this site is
2. The design: two kinds of model
3. Eleven public datasets, one schema
4. Version by version
5. What did not work
6. Labels take time
7. The results
8. Against published results and production systems
9. The machines
10. Running it yourself
11. Testing it with your own transactions
12. What it is not
13. Credits and licences

1. What this site is

This site demonstrates FinFraud, a model that screens payments for fraud. The **Score a transaction** tab takes one payment (typed in, picked from the example rows, or uploaded in a CSV file) and returns a verdict, a fraud score from 0 to 100, and a receipt that shows which fields drove the result. The **Model version** control on that tab chooses which of the eight versions scores the payment, and **Compare every version** scores the same payment with all eight at once, so the progress described below can be checked on a single payment. The **Model & training** tab shows each version's results, **Training data** describes the datasets, and **How it works** explains the models in technical detail.

The site does not train models or build data, and it keeps nothing a visitor sends: no payment is stored, and no customer history is remembered between requests. Each address can send 60 scoring requests a minute (in bursts of at most 20) and 4 file uploads a minute, so one visitor cannot occupy the server. The repository does everything the site leaves out (the training pipeline, the data build and the tools for training on rented machines), and the Running it yourself section says what that takes.

2. The design: two kinds of model

From a model's point of view, fraud comes in two kinds. Some fraud resembles fraud that has already been seen and labelled, so a bank's labelled history describes it. Other fraud resembles nothing seen before. A classifier trained on labels is the right tool for the first kind and cannot detect the second. A model of normal traffic detects the second kind but is weak on the first, because most fraud is designed to look normal. I kept both kinds, and a third model, with separate weights for each bank, decides how much to trust each one:

- **The novelty model** is a variational autoencoder (VAE) trained on legitimate payments only. It learns to compress a payment into a few numbers and to rebuild it from them; a field it rebuilds badly is unusual. Since v5 it reads the customer's previous payments first, so "unusual" means unusual *for this customer*. It never learns from a fraud label, which is why it can detect fraud that nobody has labelled yet.
- **The booster** is a gradient-boosted tree classifier (LightGBM) trained on the labelled fraud. It reads the same fields plus a block of context computed from the customer's and the merchant's past: bursts of payments in the last hour and day, how far the amount is from the customer's own average, how long the customer has been in

the data, first-time flags and, since v5, the recent history of the counterparty, device, e-mail domain, address and card.
- **The blend** is a logistic regression over the other models' calibrated scores, with an adjustment for each bank, fit on labelled validation rows. The verdict is read from the blend. For a payment from an institution the models never trained on, every version also has a *generic* mode, which scores the payment with the bank left blank.
- **The supporting models** (since v5) are a specialist booster for each bank, a sequence head on the VAE's reading of the history, a device head, an entity-graph network, text heads and behaviour verifiers. The first four each produce a calibrated score with a flag that says whether the score is present, and the blend weights each score per bank. The text and behaviour models act only when a request carries text or a session, and they add their evidence to the blend's score with a fixed weight.
- **The history model** (since v6) is a transformer with 128 million parameters, pretrained without labels to predict a customer's next payment field by field. It does not score payments itself. Its state after the customer's earlier payments and its surprise at each field of the new payment form 888 more columns for the booster (since v7, only for TabFormer and the handbook data), and since v7 a small supervised head on those columns is one more input to the blend.

v7 changed the balance between the two kinds of model. Once the booster read the history model, the VAE's two inputs to the blend cost slightly more than they added, measured as the share of fraud caught at a 1% false-positive rate, so v7's blend is fit without them. The VAE still scores every payment and keeps its own line on the receipt, with its own threshold, so the receipt can show a payment as unusual to the VAE while the verdict, which comes from the blend, is legitimate. In v7 the verdict rests entirely on the models that learnt from labelled fraud.

Every score comes with a receipt. It lists the fields the VAE could not explain and the value it expected for each, how far each field pushed the booster's score up or down, which supporting models had a score for the payment, which version scored it, and how many of the schema's fields the payment filled. The two v3 versions have no booster, so their receipts leave that part out.

3. Eleven public datasets, one schema

No single public dataset is both large and detailed, and a bank's own data looks like none of them. The corpus therefore combines eleven datasets mapped onto one schema: 42.8 million payments, 133 thousand of them labelled fraud (0.31%). Each dataset keeps its identity as a "bank" that the models know by name. Two are real, anonymised bank data (ULB/Worldline and IEEE-CIS/Vesta); the rest are simulations tuned to match real behaviour, or synthetic datasets published for research. The schema has 95 numeric and 24 categorical fields. A dataset fills the fields it can, and an unfilled field is marked unobserved, never imputed. Six entity keys link a payment to its customer's sequence and to the entity block, but never enter a model directly. A dataset with columns the schema does not cover can also have a *wide block* of those raw columns, which only the booster reads. Every dataset is split by time: the oldest 70% is for training, the next 10% is the validation split (used to stop training, calibrate the scores, fit the blend and set the thresholds), and the newest 20% is the test set that every result on this page comes from.

key	what it is	rows	fraud	licence	example rows here
fdh	Fraud Detection Handbook simulator: card payments at terminals, three fraud scenarios	1.75M	0.84%	no licence file on the data repository (the handbook's code is GPL-3.0)	no - regenerate with the handbook's notebook, or `fraudvae fetch fdh`
capone	Capital One synthetic card authorisations (recruiting dataset)	786k	1.58%	unspecified	no
tabformer	IBM TabFormer synthetic card transactions, 1991-2020	24.4M	0.12%	Apache-2.0	yes
ulb	ULB / Worldline real European card transactions, two days of 2013	285k	0.17%	DbCL v1.0	yes
paysim	PaySim mobile-money simulation	6.36M	0.13%	CC BY-SA 4.0	yes
sparkov	Sparkov simulated US card purchases with geography	1.85M	0.52%	CC0	yes
banksim	BankSim agent-based bank payments	595k	1.21%	not stated on the listing	no
ieee	IEEE-CIS / Vesta real card-not-present e-commerce (the Kaggle competition's training file)	590k	3.5%	competition rules, non-commercial	no
amlworld	IBM AMLworld HI-Small synthetic inter-bank transfers; the label is laundering	5.08M	0.10%	CDLA-Sharing-1.0	yes
baf	Bank Account Fraud (Feedzai, NeurIPS 2022): account applications with device telemetry	1.0M	1.10%	CC BY-NC-SA 4.0	yes
fraudecom	e-commerce first purchases with shared device ids	151k	9.4%	unspecified	no

None of the datasets is redistributed here. The six datasets whose licences allow it ship labelled example rows from the held-out test split: 300 rows each (150 fraud, 150 legitimate), and 179 for ULB (29 fraud). The rows are in `data/examples/` in the repository, with attribution and the licence text beside them. The example picker on the Score tab draws from these rows, and because they are test rows, the receipt can say whether the model was right. All eleven datasets are public. The GitHub-hosted ones need nothing; the Kaggle ones need a free Kaggle account, IEEE-CIS also requires joining its competition and accepting the rules, and Kaggle refuses some VPN exit addresses. The **Training data** tab links to where each dataset lives. In the repository, `fraudvae fetch` downloads what it can (the GitHub-hosted datasets directly, the Kaggle ones through the Kaggle command-line tool when it is configured, or from a zip file you put in place) and then checks every file against the recorded size and checksum of the file the models were built from. That check matters because datasets get re-uploaded and re-cut, and a different edition builds without errors but does not reproduce the results. `fraudvae examples` then produces the same kind of example file from your own copy.

4. Version by version

The table below is generated from the runs present on this server, and the text after it explains each step. "Mean AUROC, 9 shared banks" is the number that compares across versions: the mean over the nine datasets every version since v4 was trained on. "Generic" is the score with the bank left blank, which is what an institution the model never saw would get.

The table of versions is generated from the runs on the live site: finfraud.dev, Model & training. The sections below describe each version.

v3 on two CPU cores - `cpu_small-8src`

The first complete model was the VAE alone, with 0.64 million parameters. It learnt from 3.5 million legitimate rows (eight datasets, at most 500,000 training rows each) for ten epochs, in 32 minutes on two CPU cores. The fraud labels were used only on the validation split: to choose the epoch to keep, to choose between the raw and the per-field normalised score (the normalised one won), and to set each bank's thresholds. The mean test AUROC was 0.829. BankSim (0.98), ULB (0.95), Sparkov (0.94) and PaySim (0.94) were easy for a density model, because in all four the fraud makes the row itself unusual; in PaySim, for example, a fraud empties the balance in one transfer. TabFormer (0.91) and IEEE (0.75) were harder. Capital One (0.52) and the handbook data (0.66) contain fraud that looks like ordinary payments, and a model that never learns from a fraud label cannot separate it.

v3 on the full corpus - `gpu_full-4090`

The same design with 3.4 times the parameters (2.19 million) was then trained on the full training split of the same eight datasets (25.6 million rows), for 23 epochs in 18.5 minutes on a rented RTX 4090 (21.7 minutes including scoring the 7.3-million-row test split). Its mean AUROC was 0.833, against 0.829 for the two-core run. Seven times the data moved the mean by 0.004, and the validation curve was flat from epoch 9. That was the lesson of v3: the limit was the set of features, not the number of rows. The hard datasets needed labels and context, not a wider network.

v4: labels, context and the blend - `gpu_full-4090-v4`

Version 4 added three things, listed here in order of their effect. The first was the **context block**: a 1-hour burst window, the customer's own amount z-score over 30 days, tenure, first-time flags (merchant, category, cross-border), the merchant's volume, and the amount compared with the merchant's mean. All of these are causal (computed only from earlier payments) and computed when the data is built, because fraud is usually a payment that is strange *for this customer*. The second was the **booster**, trained on the labelled rows with legitimate traffic subsampled per bank and the sample weights capped so that no bank defines "normal" for the others, and early-stopped on the mean of the named-bank and bank-blanked validation AUROCs. The third was the **blend**, which replaced an earlier logistic head. AMLworld joined as a ninth dataset so that the generic mode had seen more than one kind of institution.

The VAE itself did not improve (0.826): for the third run in a row, the context features did nothing for the density model. The booster was the large step. The mean blend AUROC reached 0.916 over the nine datasets, and 0.907 over the eight v3 had, against v3's 0.833. Against the first v3 run, IEEE went from 0.75 to 0.86, TabFormer from 0.91 to 0.98 and Capital One from 0.52 to 0.77, and the share of fraud caught at a 1% false-positive rate rose from 0.1% to 35% on IEEE and from 12% to 73% on TabFormer. The generic mode went from 0.74 to 0.956. The run took 16 epochs in 17.6 minutes on the 4090, plus 8 minutes for the booster, calibration and evaluation. The booster needed three attempts before it worked; section 5 describes the two that failed.

v5: history, entities and supporting models - `v5-h200-e`

Version 5 was the largest change. The VAE gained a **history encoder**: a two-layer transformer that reads the customer's previous 32 payments into one vector, which conditions both the encoder and the decoder, so novelty is measured against the customer's own past. The schema gained the **entity block**: for the counterparty, device, e-mail domain, address and card of a payment, its volume over the last 30 days, the number of distinct customers seen with it, its age, and the fraud confirmed against it so far. That fraud count uses only labels at least 30 days old, so a row never sees its own future. Datasets with columns the schema does not cover gained a **wide block** of those raw columns, read only by the booster (431 columns for IEEE). Two datasets joined (BAF and fraudecom), and six kinds of supporting model were trained: a specialist booster for each bank, a sequence head, a device head, a GraphSAGE entity graph for each of the nine datasets with entity keys, text heads and behaviour verifiers.

It ran on a rented H200 GPU and was controlled through a relay, so the laptop could stay switched off. The raw files were uploaded once (18 GB), all eleven datasets were built in 7 minutes on 64 cores, and the full corpus was prepared in 95 seconds: 30.0 million training, 4.3 million validation and 8.6 million test rows, with 589 features. The 3.01-million-parameter VAE then trained for 30 epochs at 85 thousand rows a second, 5.8 minutes an epoch and 2.9 hours in total. A 40-minute stage followed for the booster (1,435 trees), the eleven specialists, the heads, the nine entity graphs, the text and behaviour models, calibration and evaluation. Two earlier launches of the same

configuration (runs c and d) had stopped on the wrong criterion: the per-epoch VAE AUROC, which swings by up to 0.3 on single datasets because it scores with one posterior sample over small sets of fraud. Run e stops on the held-out reconstruction objective instead, which changes smoothly from epoch to epoch.

On the nine datasets v4 had, the blend went from 0.916 to 0.936, and the gains came where the new columns were. Capital One went from 0.765 to 0.838 (from 8% to 21% of its fraud caught at a 1% false-positive rate) and IEEE from 0.858 to 0.940 (from 35% to 56%). The ablation traced these gains to the delayed-label entity features for Capital One and IEEE, and to the wide block for IEEE. The history-conditioned VAE became a better novelty model on the banks with customer sequences (Capital One 0.51 to 0.61, IEEE 0.73 to 0.79, AMLworld 0.82 to 0.91) and a worse one on the simulators (PaySim 0.94 to 0.80). A PaySim fraud empties the balance in one transfer; on its own that row is an outlier, but a model that has read the account's previous rows already expects part of it. The blend gives the VAE's score a weight near zero on most banks; the VAE's job is the receipt and the fraud nobody has labelled.

v5: one-week fraud labels - `v5-h200-f`

Run e left the handbook data at 0.69, where every earlier model had left it, and the reason was mechanical. The simulator's second fraud scenario compromises a terminal for 28 days, and the schema's fraud-history features count only labels at least 30 days old, so by the time a terminal's fraud is "known", the terminal is clean again. The handbook's own baseline uses a one-week label delay for exactly these terminal-risk features. Whether a week is realistic depends on how fast a deployment receives its labels, not on the model, so v5 carries both: the 30-day features in the schema for the VAE and the booster, and the same features at a 7-day delay in the wide block for the booster only, for every dataset with entities. The VAE did not change. The entity features were rebuilt and the booster side was refit on a copy of e's checkpoint, which took 30 minutes on the H200 box. The handbook data went from 0.691 to 0.893 (from 34% to 75% of its fraud caught at a 1% false-positive rate), IEEE from 0.940 to 0.958, the nine-bank mean from 0.936 to 0.961 and the generic mode from 0.953 to 0.966.

v5: recent-fraud windows and label delay as a setting - `v5-h200-h` (what shipped until v7)

Run h made two more changes to the booster side, on the same checkpoint. **Recent-fraud windows** add the one-week fraud count and share over the entity's payments in the 7 and 30 days before the label horizon, so a terminal compromised this month is not diluted by years of clean history. **Label-latency dropout** blanks the whole one-week block on a random 30% of the booster's training rows, so a deployment whose labels take a month is a case the model has seen in training, not an input it has never met. The score API takes `label_delay_days` (the "labels arrive within" control on the Score tab), and every version since is evaluated twice, once for each latency. The booster stage took 41 minutes. TabFormer went from 0.986 to 0.994, and from 74% to 86% of its fraud caught at a 1% false-positive rate, because card fraud in that file comes in bursts and a lifetime fraud share had hidden each month's burst under years of clean history. The handbook data went from 0.893 to 0.905. Scored as a deployment whose labels take a month, the same model is within 0.001 of run e on every bank. **Results: 0.963 mean over the nine shared banks, 0.928 over all eleven, 0.969 generic.**

v6: a pretrained model of payment histories - `v6-base-e`

v5 read a customer's past through a small transformer inside the VAE, and giving that reading to the booster changed nothing (run i, in section 5). v6 tested the same idea at a much larger size. A **transaction encoder** with 128 million parameters (width 768; four layers read the fields of one payment and twelve read up to 128 payments in order) was trained without any labels to predict a customer's next payment field by field. Every field of the schema becomes a token: a numeric field becomes one of 64 buckets, a categorical field its own value, and the gap since the previous payment one of 40 buckets. The vocabulary has 6,389 tokens over 121 fields, and each dataset uses only the fields it fills. The encoder's corpus is sixteen datasets: the eleven, plus five more AMLworld variants that add rows for pretraining but are not part of the labelled corpus. That makes 469 million rows, 328 million of them used for training.

It ran on a rented node with eight B200 GPUs. The sixteen-dataset set was prepared in 457 seconds, and the 60,000 training steps took four and a half hours and covered 100.8 billion tokens. The validation loss on the next payment ended at 1.25 nats per field; a uniform guess over a numeric field's 64 buckets would score ln 64 = 4.2. The encoder itself scores nothing. For every row it produces 888 numbers: its 768-number state after the customer's earlier payments, its surprise at each of the 119 fields of the new payment, and the mean of those surprises. The booster side of e's checkpoint was then refit with these columns added to its own: computing them for the 42.8 million rows took 13 minutes on seven GPUs, and the booster stage took 72 minutes.

v6 traded gains on three banks for small losses on three others, and it stays on the site as the record of that trade. TabFormer, with thirty years of long card histories and the dataset closest to the encoder's own training data, went from 86% to 94% of its fraud caught at a 1% false-positive rate, and from 73% to 88% when labels take a month: the history model reads payments, not labels, so late labels cost it nothing. The handbook data and AMLworld gained slightly (0.905 to 0.908; 85% to 88% of fraud caught at 1%). Capital One, IEEE and fraudecom lost slightly (0.841 to 0.836; 0.958 to 0.956; 10% to 7% of fraud caught at 1%), and in each case the paired interval on the same test rows excludes zero. Over the eleven datasets the mean moved from 0.9277 to 0.9282, and over the nine shared banks it did not move (0.963). The shorter pretraining runs on the way showed the same pattern: more steps kept lowering the encoder's own loss and stopped improving the detector.

v7: the history model for two banks only - `v7-f` (what ships)

v7 started with measurement tools, because v6's differences were a few thousandths and the results tables could not show which of them were real. `fraudvae compare` scores two versions on the same test rows and resamples those rows a thousand times, so every difference comes with an interval, per bank and under both label settings. The headline measure moved from the mean AUROC, which three saturated simulators hold still, to the share of fraud caught at a 1% false-positive rate. Seven variants of the booster side then ran in one night on a rented box with two H200 GPUs, and each was compared with h and with v6 on all 8.57 million test rows.

A control experiment explained v6's losses. With the block's rows shuffled within each bank (the same 888 columns, carrying no information), Capital One and IEEE lost as much as the real block had cost them. The losses therefore came from adding 888 columns to the trees, not from anything the encoder read. Compressing the block to 64 principal components did not avoid the losses and gave back most of TabFormer's gain. **Routing** did avoid them: the booster gets the block only for TabFormer and the handbook data, and Capital One and IEEE are level with h again. On top of that, a small **supervised head** on the block, trained on the labelled rows of the two routed banks, became one more input to the blend, and the **blend was fit without the VAE's two columns**.

Against h on the same rows, the mean share of fraud caught at a 1% false-positive rate is 0.011 higher (interval 0.009 to 0.013), and 0.018 higher (interval 0.016 to 0.021) when labels take a month. TabFormer catches 95% of its fraud at that rate instead of 86%, and 91% instead of 73% with month-old labels; the handbook data catches 79% instead of 78%, and AMLworld 86% instead of 85%. No bank is worse except PaySim, by a few millionths of AUROC at its ceiling. **Results: 0.964 mean over the nine shared banks, 0.928 over all eleven, 0.971 generic.** The rule written into the plan before the experiments was that no bank may be worse by a paired interval under either label setting. Under that rule PaySim's few millionths count as a loss and nothing would ship. I judged the results with a minimum effect size instead, under which v7-f has no loss and the largest gain, and shipped it.

The history has to arrive with the payment. v6 and v7 were both measured with every customer's earlier payments in place, and the first test on the site's own example rows showed what happens without them. A payment that arrives alone looks to the encoder like a customer's first payment. The trees then see a new customer's state next to fields that count thousands of earlier payments, and they raise false alarms: on 43 of TabFormer's 150 legitimate example rows for v6 and on 13 for v7, against 1 for h. The example rows now carry their customer's last eight payments, which brings both versions back to their measured behaviour on those rows: 1 false alarm each, and 141 (v6) and 142 (v7) of the 150 frauds caught, against 133 for h. When a payment's own fields count earlier payments but the payments themselves are missing, v7 scores it with the history model's columns left blank, a case its trees met in training, instead of reading it as a customer's first payment. v6's trees never met a payment without the block, so v6 keeps the first-payment reading, and its receipt says that the payment arrived alone.

5. What did not work

The failed attempts shaped the model as much as the successful ones, so they are recorded here. None of them can be loaded on this site; the training notes give the full accounts.

- **More rows for the VAE.** Three times (v3, v4, and v5's capped probes against the full runs), seven times as many legitimate rows changed the density model by a few thousandths. The booster's quality was set by the features and the labels. Its training set is 200,000 legitimate rows per bank plus every fraud row, whichever prepared set it is drawn from, so a laptop trains the same booster.
- **Per-field surprises in the blend.** The first v4 blend also took the 81 per-field surprise z-scores and the latent mean. It scored well on the validation split (TabFormer 0.996) and badly on the test split (0.839, below both models alone). The surprises are unbounded, the newest 20% of a 30-year file is in a different regime (one column reached z≈1000), and a slightly negative pooled weight let that one column outweigh the booster. The blend now sees only bounded, calibrated scores, and the surprises stay on the receipt as explanation.

- **Uncapped sample weights.** Subsampled legitimate rows were weighted back to each bank's true base rate. On the full corpus that weight was 85 for TabFormer and 1 for ULB, so in the bank-blanked mode one bank's traffic defined "normal" (generic 0.936 instead of 0.957 on identical rows). The weight is now capped at 3.
- **Three times the legitimate rows per bank in the booster** (600,000 instead of 200,000, with a higher learning rate). The booster learnt patterns specific to one period of time rather than to fraud: a gender column became the third most important feature by gain, and the results on the newest months were worse than the two-core baseline's.
- **Lifetime entity counts.** The first v5 entity block included "distinct customers ever seen with this entity" and "days since first seen". On a 30-year file these grow with the calendar. The booster used them to tell the era, the newest months were an era it had never seen, and TabFormer lost 0.05 AUROC and 57 points of recall. Windowed and capped versions replaced them (30-day counts, ages capped at a year): TabFormer returned to 0.977 and nothing else was lost.
- **bf16 mixed precision with the history transformer.** The gradient norm overflowed around step 700 of the first epoch at every learning rate tried, although each individual gradient was finite, and a few steps later the embeddings were NaN. fp32 was 10% slower and stable, because the bottleneck is gathering the history windows on the CPU, not the matrix multiplications. Since then the inputs are bounded at ±20σ, and a step with a non-finite value is skipped instead of applied.
- **Stopping on the per-epoch VAE AUROC** (runs c and d). This is a one-sample estimate over small sets of fraud, and it swung by 0.3 between neighbouring epochs. Run e stops on the held-out reconstruction objective.
- **Unregularised per-bank blend weights.** ULB has 21 fraud rows in the half of the validation split the blend is fit on, and the ULB interaction terms memorised them (0.915 on the held-out half, against the booster's 0.953). Hierarchical shrinkage, which scales each bank's interaction columns by √(fraud rows / 10,000), pulled ULB onto the shared weights (0.953) and left the large banks unchanged.
- **The history encoder's read-out as a booster input** (run i). It changed nothing: 0.9627 against h's 0.9634 over the nine banks. The trees reached the same validation AUROC in fewer rounds, so the 128 columns do carry information, but the velocity and entity blocks already contain it. The option stays in the code as a documented negative result.
- **A longer VAE** (run g: a 48-payment window, 40 epochs) was stopped at epoch 7 once h's results were in. The blend gives the novelty score a weight of about zero, so a better novelty model would have changed the receipt and nothing the verdict depends on.
- **The history model for every bank** (v6 as it was) made Capital One, IEEE and fraudecom slightly worse, each by an interval that excludes zero. A control with the block's rows shuffled within each bank reproduced the losses, so they came from adding 888 columns to the trees. v7 routes the block only to the two banks where it improves the result.
- **The block compressed to 64 components** (v7-c) kept the losses and little of the gain: +0.031 of TabFormer's fraud caught at a 1% false-positive rate, against +0.080 for the raw block.
- **Routing the block to IEEE as well** (v7-b) cost IEEE 0.010 of its fraud caught at a 1% false-positive rate, and the interval excludes zero. The encoder adds nothing for IEEE.
- **Fine-tuning the encoder on the fraud labels** (v7-g). The last two layers were unfrozen for one pass over 1.12 million labelled windows, which took 48 minutes. The encoder's own head improved on all four banks it saw. The full model rebuilt on it gained on TabFormer with late labels and lost on IEEE under both settings, and the mean did not move. One pass pulled the top layers towards the bank with most of the labels.
- **More pretraining steps.** Between 10,000 and 60,000 steps the encoder's loss on the next payment kept falling, but the detector built on it did not improve: a probe on its state for TabFormer moved from 0.966 to 0.973, with an interval that includes zero. A second, longer pretraining depended on that test, so it was not run.
- **Infrastructure.** LightGBM with unlimited threads on a 256-thread host ran five times slower than on two cores; it now limits itself to the cores it is allowed. A pandas datetime conversion of 24 million rows exhausted a 6 GB build container twice. Microsecond timestamps were divided by the wrong constant until the velocity features came out impossible. A sorted memmap read returned rows in a different order from the labels they were paired with. That was the real cause of a generic mode at 0.50, which had first been put down to a limitation of the model.

6. Labels take time

A fraud label is not known when the payment happens. A chargeback arrives weeks later, a customer's complaint days later, and a compromised terminal is noticed only once a pattern appears. Any feature of the form "how much confirmed fraud has this entity had" must therefore describe the past *as it was known at the time*, and a model trained with labels it would not have had in production overstates its accuracy. The schema's entity block assumes that labels

take 30 days and the booster's one-week block assumes 7; the latency dropout trained the booster for both. On the Score tab, "labels arrive within" tells the model which deployment it is in. Past a week, the one-week block is blanked before the booster sees it, and the receipt says so ("used", "blanked" or "not sent"). Every version's results table shows both settings. For h the two numbers are 0.963 (nine-bank mean, labels within a week) and 0.935 (the same rows, labels within a month). The second is within 0.001 of where the model stood before the one-week block existed (run e, 0.936), which is the drop to expect when the fresher labels are taken away. For the shipped model the two numbers are 0.964 and 0.937. The difference from h is the part of a score that does not depend on labels: the history model reads payments, so TabFormer keeps 91% of its fraud caught at a 1% false-positive rate when labels are a month late, where h kept 73%.

7. The results

These are the shipped version's results on the held-out test split: the newest 20% of every dataset, 8.57 million rows of which 27,303 are fraud, scored with thresholds fixed on the validation split. **AUROC** is the probability that a randomly chosen fraud row scores above a randomly chosen legitimate one: 0.5 is a coin flip and 1.0 is perfect separation. **r@1%** is the share of fraud caught when the alert threshold flags 1% of legitimate payments, which is closer to how a fraud team experiences a model. The "30-day labels" column scores the same model as a deployment whose labels take a month. The last two columns are h, the version that shipped before v7, on the same rows.

source	VAE	booster	seq	local	history head	device	graph	blend	r@1%	30-day labels: blend / r@1%	h blend	h r@1%
fdh	0.624	0.903	0.656	0.907	0.693	–	0.501	**0.907**	79%	0.686 / 35%	0.905	78%
capone	0.613	0.837	0.815	0.839	–	–	0.767	**0.841**	21%	0.838 / 20%	0.841	21%
tabformer	0.940	0.996	0.985	0.998	0.974	–	0.815	**0.998**	95%	0.996 / 91%	0.994	86%
ulb	0.962	0.986	0.987	0.985	–	–	–	**0.985**	89%	0.985 / 89%	0.984	89%
paysim	0.798	1.000	0.991	1.000	–	–	0.554	**1.000**	100%	1.000 / 100%	1.000	100%
sparkov	0.862	1.000	0.996	1.000	–	–	0.953	**1.000**	100%	1.000 / 100%	1.000	100%
banksim	0.990	1.000	0.999	1.000	–	–	0.998	**1.000**	99%	0.999 / 99%	1.000	99%
ieee	0.795	0.957	0.904	0.955	–	0.733	0.825	**0.958**	63%	0.940 / 56%	0.958	63%
amlworld	0.912	0.990	0.984	0.990	–	–	0.846	**0.990**	86%	0.990 / 86%	0.990	85%
baf	0.658	0.892	0.812	0.891	–	0.768	–	**0.892**	26%	0.892 / 26%	0.893	25%
fraudecom	0.643	0.643	0.644	0.639	–	0.639	0.516	**0.641**	10%	0.641 / 9%	0.641	10%
mean, 11 sources	0.800	0.928	0.889	0.928	0.833	0.714	0.753	**0.928**		0.906	0.928	
mean, v4's 9 sources	0.833	0.963						**0.964**		0.937	0.963	
generic (bank unknown)	0.625	0.970	0.930	–	–	0.737	–	**0.971**	72%	0.955 / 64%	0.969	70%

`seq` is the sequence head on the VAE's history read-out, `local` the bank's own specialist booster, `history head` the supervised head on the pretrained history model's columns for the two banks they are routed to, and `device` and `graph` the device and entity-graph heads, for the datasets that have those columns. The mean over the eleven datasets barely moves (0.9277 to 0.9283) because eight of them were already at their current level; the rows that changed are TabFormer, the handbook data and AMLworld. For comparison, v4's nine-bank mean was 0.916. The supporting models trained on their own public data have their own results: AUROC 0.994 for the URL head, 0.972 for the free-text head and 0.783 for the profile head; equal error rates of 0.11 (keystroke), 0.12 (motion), 0.17 (touch) and 0.24 (mouse) for the behaviour verifiers; and AUROC 0.78, 0.88, 0.92 and 0.98 for the graph encoder on the DGraph-Fin, Elliptic, YelpChi and Amazon benchmarks.

How to read the table: the simulators saturate because their fraud follows rules a model can learn, which says nothing about real fraud. The real and realistic datasets are where the versions differ, and the hard ones are hard for identifiable reasons. Fraudecom's fraud is defined by a shared device, and 95% of its test rows come from a device seen for the first time. BAF consists of account applications with no customer history; v7 catches 55% of its fraud at a 5% false-positive rate, which is the range the dataset's own paper reports for gradient-boosted trees. Capital One's fraud is ordinary-looking purchases whose only strong public signal is a rare CVV mismatch.

8. Against published results and production systems

Where a published result exists on comparable data, the shipped model matches or exceeds it. ULB scores 0.985 against the 0.95–0.98 reported in the gradient-boosting literature. The handbook data scores 0.907 against the handbook's own XGBoost baseline of about 0.87 with the same one-week assumption. BAF has 55% of its fraud caught at a 5% false-positive rate, against "about half" in the dataset's paper. IEEE scores 0.958 on a time split of the competition's training file, which is in the range of the competition leaderboard; the leaderboard used a different split and a hidden test set, so this is not a ranking. On the simulators every method saturates, so they allow no comparison.

A production system has several things this project lacks, and a fair comparison has to name them. It has features that separate ordinary-looking fraud: device fingerprints, data shared across institutions, and account-age and terminal signals that the public datasets do not carry. It retrains continuously as the fraud and its labels change. It answers in under 50 milliseconds at thousands of payments a second, while this server scores one payment in 0.3 to 0.8 seconds on CPU (the upper end when the history model reads the customer's last eight payments), and the v3 VAE alone scores about 90 thousand rows a second in batch on the laptop's CPU. It also sits inside case-management tools, feedback loops and regulatory review. The architecture here follows how the industry builds these systems: a supervised model on causal context features, a novelty model beside it, per-segment calibration, an explanation for every score, and label latency treated as an input. What separates this project from a production system is the realism of the data, not the modelling. It is a demonstration built in the open on public data, and its results are accurate for that setting.

9. The machines

The table lists every machine involved, what it did and what hardware it had. The hardware matters because the question "would this be better on a bigger machine?" has a precise answer, given in the next section.

machine	what it did	hardware
the sandbox	Development and tests throughout; the v3 capped-corpus run (`cpu_small-8src`: 10 epochs, 32.5 min); the v4 capped baseline (`cpu_small-9src-v4`: 9 epochs, 33.6 min, then 25 min of booster, calibration and evaluation); the v5 development probes on two- and four-dataset capped sets.	A Linux cloud container: 2 x86-64 CPU cores, under 8 GB of RAM, no GPU. Python 3.13, PyTorch 2.14 (CPU).
the laptop	The app and the Compute tab that rented and drove the GPU boxes; the full raw-to-processed build of the corpus (2.6 GB of parquet); the laptop timings quoted on this page (about 170 ms per scored transaction with the v5 model on the Apple GPU).	A MacBook Air with an Apple M3 chip and 24 GB of unified memory, macOS 26.6, PyTorch with MPS.
the 4090 box	The v3 full-corpus run (23 epochs, 18.5 min; 21.7 min end to end) and the v4 run (16 epochs, 17.6 min, plus 8.4 min for the booster, calibration and evaluation), each provisioned from the laptop's Compute tab: 1.0-1.4 GB of processed parquet uploaded, `prepare` on the box in 71 s.	1× NVIDIA GeForce RTX 4090 (24 GB), 16 CPU cores allotted on a shared 256-thread host, rented by the hour (vast.ai). PyTorch 2.14.1+cu132.
the box (H200)	All of v5, with the laptop off, through the relay: the raw files uploaded once (18 GB), the eleven-dataset build (7 min), the full `prepare` (95 s), run e (30 epochs, 171.8 min, then about 40 min for the booster side), the f and h refits (29.8 and 40.6 min), the supporting models' training, the booster sweep and ablation, and the failed runs i and g.	1× NVIDIA H200 NVL (141 GB), 64 CPU cores, 188 GB of RAM, rented by the hour. PyTorch 2.14.1+cu130, fp32.
the node (8× B200)	All of v6, in about ten hours: the sixteen-dataset build and both prepared sets (the eleven-dataset one in 39 s, the sixteen-dataset one in 457 s), a 10,000-step trial of the encoder and its booster round, the 60,000-step pretraining (four and a half hours), the block computed for every row, and the booster rounds on the 32,000- and 60,000-step encoders.	8× NVIDIA B200, 192 CPU cores, 2 TB of RAM, rented by the hour (vast.ai). Python 3.12, PyTorch 2.14.1+cu132.
the box (2× H200)	All of v7 in one night of eleven hours: the corpus prepared again and checked against the node's counts, both baselines re-evaluated to their published results, the 60,000-step encoder's block computed for every row, seven booster-side stages two at a time, the probes, the fine-tuning, and nineteen paired comparisons under two label settings each.	2× NVIDIA H200 NVL, 350 GB of RAM, rented by the hour (vast.ai).
this server	The hosted demonstration: inference only, every version loaded, rate-limited.	A VPS at 1984.hosting (Iceland): 6 vCPU, 16 GB of RAM, 320 GB SSD, no GPU.

10. Running it yourself

The repository trains everything on this page from the public downloads; nothing here needed private data. Two facts about hardware come first, because they decide what is worth doing:

A faster machine gives you the same model sooner, not a better one. The configuration that ships (`gpu_full`) is fixed: the same data, the same model size and the same number of epochs. On a slower GPU or a CPU it takes longer and reaches the same result, within the run-to-run noise of training. Hardware changed the result only through *choosing a different configuration*. The `cpu_small` configuration is a smaller model (0.64M parameters against 2.2–3.0M) on a capped corpus: the two-core run kept at most 500,000 training rows per dataset and finished in 32 minutes, and the README's recipe, with 1.5 million rows per dataset, takes about an hour on a laptop CPU. For v3 the two configurations were nearly identical (0.829 against 0.833). For v4 the capped CPU baseline scored 0.003

below the full run (0.913 against 0.916), because the booster's training set is the same size either way. For v5 the history encoder needs the long, real customer sequences of the full corpus, so a capped CPU run of v5 would score clearly below the results above. It was used only for development probes and never trained to completion.

what	on a laptop (CPU / Apple GPU)	on one RTX 4090	on one H200 / H100
download the eleven raw datasets (`fraudvae fetch`)	about 15 GB of downloads (the AMLworld archive alone is 8 GB, of which one file is used), about 60 GB unpacked with the supporting models' datasets; minutes to hours depending on the connection; eight of the datasets need Kaggle credentials		
build the corpus (`fraudvae build`)	1-3 hours (estimate; the full build ran on the laptop, because the adapters stream the big files in chunks)	-	7 min on 64 cores
prepare the training arrays (`fraudvae prepare`)	10-20 min (estimate)	71 s (v4 schema)	95 s
v3 / v4 VAE, `cpu_small` (capped corpus)	32 min measured on two CPU cores at 500,000 rows per dataset; about an hour at the README's 1.5 million	a few minutes	a few minutes
v4 VAE, `gpu_full` (full corpus)	about 2 days (measured at 9,800 rows/s on the laptop CPU: about 50 h for 30 epochs, and early stopping ends it at about half)	17.6 min	under 10 min (estimate)
v5 VAE, `gpu_full` (history encoder, full corpus)	several days (estimate; gathering the history windows is CPU-bound on every machine)	about 1 day (estimate; 24 GB of memory forces smaller batches)	2.9 h
the booster side (`fraudvae post`: booster, specialists, heads, graphs, calibration, evaluation)	1-3 hours (estimate; the booster itself is CPU work on 200,000 rows per bank and does not need a GPU)	about 25 min	30-40 min
the v6 encoder (`fraudvae pretrain`, 128M parameters, sixteen datasets)	not practical; the laptop runs the 1.9M-parameter smoke configuration in six and a half minutes to check that the training loop works	not tried	4.5 hours on eight B200s
the booster side on a pretrained encoder (`fraudvae txfm-cache`, then `fraudvae post`)	not tried	not tried	v6: 13 min of block computation on seven B200s, then 72 min; v7: 91 min on one H200 with the block already computed

The GPU time for the v3 and v4 runs on the 4090 cost a few dollars in total. The shipped v5 run (e plus the f and h refits) used about $16 of H200 time, and the whole v5 campaign, including failed runs, supporting-model training and ablations, about $105. v7's night on the two-H200 box cost $101 for seven variants, the paired comparisons and the fine-tuning. The one expensive stage is v6's pretraining, four and a half hours on an eight-GPU node, and it does not need repeating, because the pretrained encoder is included in the v6 and v7 bundles. The Compute tab in the repository's app benchmarks the real model on your machine and gives this estimate before you rent anything.

You do not have to train anything. Every version on this site is published as a pretrained bundle on the repository's Releases page, and `fraudvae models pull v7-f` (or any other name from the timeline) puts it in `runs/`, where the app loads it. The bundles from v5 on leave out one file: the PaySim entity graph (822 MB per run), whose head scored 0.55 on PaySim and has no weight in the blend. The server scores PaySim without it, and the receipt says so. In short:

```
git clone https://github.com/deanOcoin/fraudvae && cd fraudvae
python3 -m venv .venv && source .venv/bin/activate && pip install -e ".[dev]"
fraudvae models pull v7-f               # the shipped version, 845 MB (the pretrained encoder is
513 MB of it)
fraudvae serve                          # http://127.0.0.1:8765 - the same app as this site,
with training enabled

# or train it yourself
fraudvae fetch                          # downloads what it can, tells you what needs Kaggle
credentials
fraudvae build && fraudvae prepare --max-rows-per-source 1500000
fraudvae train --config cpu_small      # about an hour on a laptop CPU, the v3/v4-sized model
fraudvae train --config gpu_full       # the shipped configuration; see the table above for what
it takes
```

Requirements: Python 3.10+, PyTorch 2.4+ and LightGBM 4+. The full corpus needs 60 GB of disk for raw plus processed data, and 32 GB of RAM for the v4 prepare step (64 GB or more for v5's). Inference alone runs on any machine with 4 GB of free RAM: it loads a v5 version in a few seconds and scores a payment in a fraction of a second. A v6 or v7 version also holds the encoder, which adds half a gigabyte, and on a laptop it reads a payment with eight earlier ones in under a fifth of a second. The README explains the macOS OpenMP setting that otherwise crashes the booster on its first call.

11. Testing it with your own transactions

There are three ways, in increasing order of effort:

- **Example rows.** The Score tab's example picker draws labelled rows from the held-out test split of the six datasets that may be redistributed. Pick a dataset and fraud or legitimate, and the receipt shows what the model made of the row and whether it was right; the label is shown next to the verdict. Any version can score the same row. Where the dataset has customers, the row brings its customer's last eight payments with it in the history box, because every version since v5 judges a payment against the customer's own past and v6 and v7 were measured that way.
- **A payment you type in.** Fill in what you know (the amount, the hour, the channel, whether this is the customer's first time at this merchant, the device, anything else the form offers) and leave the rest blank. Blank fields are unobserved, not zero, and the model was trained to score with partial information. Choose the bank whose normal traffic it should be judged against, or *generic* for a payment from none of the eleven; generic is the model's answer for an institution it never saw. A customer history can be pasted in as previous payments, so that the versions from v5 on have a sequence to read; v6 and v7 read up to 127 earlier payments. When a payment's fields count earlier payments that were not sent, v7 scores it without the history model, and the receipt says so.
- **Your own CSV file.** The batch upload on the Score tab takes a CSV file with any subset of the schema's columns (the column reference is on the Training data tab) and scores up to 5,000 rows per upload on this site; the app on your own machine has no such limit. If the file has an `is_fraud` column, the summary reports AUROC and recall on it, so by running the app locally you can evaluate a version on your own data without sending it anywhere. A file's rows arrive without histories, so v6 and v7 show their gain on single payments with a history, not on a file. On this site, at most 1,000 rows of a file go through the encoder.

The server keeps none of this. In public mode the customer store is read-only and empty, requests are logged only in nginx's access log, and the rate limit is per address.

12. What it is not

It is not a product, and it makes no claim about real-world fraud rates. The corpus is public and mostly simulated, its labels are the datasets' own labels, and the real datasets are anonymised to the point where many signals a bank would have are missing. A model trained on it scores a real payment by analogy with the closest bank it knows. That demonstrates the method; it is not a fraud decision anyone should act on. The specific limits are listed in the last

section of the **How it works** tab and in the training notes: ordinary-looking fraud without terminal or device signals, devices seen for the first time, account applications with no history, and anything that a simulated card file ending in 2020 cannot know about 2026.

13. Credits and licences

The code is MIT-licensed. The datasets keep their own licences, listed above and in `docs/DATA_SOURCES.md`, and the citation for each is in `fraudvae/data/registry.py`. The supporting models' training sets are also public and credited in the repository: malicious URLs, scam job postings, bot profiles, the Balabit, BB-MAS and HMOG behaviour sets, and the DGraph-Fin, Elliptic, YelpChi and Amazon graph benchmarks. The ideas are established ones (heterogeneous-likelihood VAEs, gradient boosting on causal velocity features, per-segment logistic calibration, delayed-label features), and the training notes say where each came from. This project's own contribution is the combination: the eleven-dataset schema, the treatment of label latency and the receipts. It was built in the open by Dean Mostafa, and the repository's history records how. A large part of the code was written with the help of an Anthropic Claude model, more of the front end than of the training side. The ideas, the design of the models and the data, and every decision about what to build and what to leave out are mine, not the model's.